TRABAJO PRÁCTICO INTEGRADOR

Diseño e implementación de una base de datos NoSQL

MongoDB · API · Repositorios · Análisis de datos · Docker Compose

Dato	Descripción
Asignatura	Base de Datos Aplicada
Duración	4 semanas
Modalidad	Equipos de 3 o 4 integrantes

Propósito

Construir una solución operativa completa, que utilice MongoDB como base de datos documental, exponga operaciones mediante una API y permita analizar los datos almacenados.

1. Fundamentación

Las bases de datos NoSQL constituyen una alternativa relevante para aplicaciones que administran información heterogénea, estructuras que evolucionan y sistemas distribuidos. En este trabajo se aplicarán los conceptos del modelo documental mediante MongoDB, considerando sus posibilidades y las decisiones de diseño que requiere.

2. Objetivos de aprendizaje

- Reconocer cuándo puede resultar conveniente una base de datos documental.
- Diseñar colecciones y documentos de acuerdo con los requerimientos y patrones de acceso.
- Implementar y poblar una base de datos MongoDB con datos representativos.
- Construir consultas, actualizaciones y agregaciones que respondan a preguntas del dominio.
- Integrar MongoDB con una API mediante una capa de repositorios.
- Crear índices y analizar su uso en consultas relevantes.
- Identificar necesidades de consistencia y justificar operaciones atómicas o transacciones.
- Analizar los datos almacenados mediante Python u otra herramienta apropiada.
- Entregar una solución reproducible, documentada y verificable.

3. Consigna general

Cada equipo deberá diseñar e implementar una solución de base de datos NoSQL utilizando MongoDB para dar soporte a una aplicación operativa. La solución deberá abordar una problemática concreta, almacenar información persistente, permitir administrar la entidad principal mediante operaciones CRUD y ofrecer consultas que respondan a necesidades reales del dominio.

La base deberá integrarse con una API a través de una capa de repositorios y ejecutarse en un entorno reproducible con Docker Compose. Finalmente, se desarrollará un módulo de análisis que produzca indicadores, visualizaciones y conclusiones a partir de los datos almacenados.

El objetivo no es solamente demostrar que MongoDB funciona, sino justificar la estructura documental, las consultas, los índices y las estrategias de consistencia elegidas.

4. Selección de la problemática

Cada equipo podrá proponer un dominio propio, sujeto a aprobación docente. La problemática deberá ser suficientemente rica para justificar el uso de una base documental, pero acotada para completarse en cuatro semanas.

Dominio posible	Alcance orientativo
Catálogo y gestión de productos	Productos con atributos variables, categorías, proveedores, etiquetas, stock y precios.
Plataforma académica	Cursos, docentes, estudiantes, inscripciones y actividades.

Sistema de biblioteca	Libros con metadatos heterogéneos, autores, ejemplares, préstamos y usuarios.
Gestión de eventos	Eventos, sedes, participantes, inscripciones y actividades.
Gestión de pedidos	Clientes, pedidos, productos, importes, estados y entregas.
Otros dominios	Turnos, publicaciones, turismo, videojuegos o administración de proyectos.

Condición: la propuesta debe tener una entidad principal y, como mínimo, una colección adicional cuando el dominio lo justifique. No se aceptarán proyectos limitados a almacenar documentos sin operaciones, consultas ni análisis.

5. Requerimientos del proyecto

5.1. Análisis y diseño de la base de datos

- Describir el problema y los usuarios o actores involucrados.
- Definir los requerimientos funcionales principales.
- Identificar las entidades o conceptos relevantes.
- Proponer las colecciones y la estructura de documentos de ejemplo en JSON/BSON.
- Identificar atributos obligatorios y opcionales, documentos anidados y arreglos.
- Describir las relaciones entre documentos y justificar las decisiones de modelado.

Se deberá justificar cuándo conviene embeber documentos y cuándo utilizar referencias. La decisión deberá responder a los patrones de acceso y actualización previstos, no limitarse a copiar un modelo relacional. Como mínimo se espera un modelo de dos colecciones relacionadas conceptualmente, salvo aprobación docente de una estructura diferente.

Entregable: documento de diseño con colecciones, ejemplos de documentos, relaciones y justificación.

5.2. Implementación y carga de datos

- Crear las colecciones necesarias.
- Preparar scripts reproducibles para cargar los datos de prueba.
- Verificar la cantidad y consistencia básica de los documentos.
- Incluir datos variados, atributos opcionales y algunos casos límite.
- Documentar cómo reinicializar la base y repetir las pruebas.

Se recomienda un mínimo de 100 documentos distribuidos entre las colecciones y, cuando resulte razonable, entre 200 y 500. No se requiere una prueba de carga de millones de registros: se evaluará la calidad del diseño y de las consultas.

Entregable: scripts de creación y carga, archivos de datos si corresponde y evidencia de ejecución.

5.3. Consultas MongoDB

Se deberán implementar al menos 10 consultas diferentes, documentando para cada una su objetivo, comando y resultado esperado.

- Dos consultas con filtros y operadores de comparación.
- Una consulta que combine condiciones mediante operadores lógicos.
- Una consulta sobre atributos anidados.
- Una consulta sobre arreglos o listas.
- Una consulta con proyección, ordenamiento y límite de resultados.
- Dos consultas que utilicen el pipeline de agregación mediante aggregate().
- Una consulta con agrupamiento y cálculos estadísticos.
- Una consulta adicional de complejidad elegida por el equipo.

Las consultas deben responder preguntas útiles del dominio. Variar únicamente el valor de un filtro no se considerará suficiente para cubrir tipos de consultas diferentes.

Entregable: archivo de consultas comentado y evidencia de ejecución.

5.4. API, repositorios y operaciones CRUD

La aplicación deberá administrar la entidad principal mediante una API. Se puede utilizar el lenguaje y framework que el equipo ya domine. Python con FastAPI o Flask es una alternativa recomendada, pero no obligatoria si el Trabajo Final ya utiliza otra tecnología.

Método	Endpoint de referencia	Función
POST	/entidades	Crear una entidad
GET	/entidades/:id	Obtener una entidad por ID
PUT o PATCH	/entidades/:id	Actualizar una entidad
DELETE	/entidades/:id	Eliminar una entidad
GET	/entidades	Listar entidades con filtros básicos

El recurso /entidades deberá reemplazarse por el nombre correspondiente al dominio.

- Implementar una capa de repositorios que concentre el acceso a MongoDB.
- Validar los datos recibidos y manejar identificadores inexistentes.
- Devolver códigos HTTP y mensajes de error adecuados.
- Separar razonablemente API, lógica de negocio y acceso a datos.
- Permitir demostrar las operaciones mediante Swagger, Postman, Insomnia o herramientas equivalentes.

Entregable: código de la API y los repositorios, instrucciones de ejecución y pruebas de endpoints.

5.5. Índices y análisis de rendimiento

- Crear al menos dos índices adecuados a las consultas implementadas.
- Explicar qué campos se indexan y por qué.
- Ejecutar consultas relevantes con explain("executionStats").
- Comentar los resultados y el uso observado de los índices.
- Distinguir el posible beneficio de un índice de su costo de almacenamiento y mantenimiento.

No se exige demostrar mejoras de tiempo significativas con pocos datos. Se evaluará la pertinencia de los índices y la interpretación de los resultados. La búsqueda de texto o geoespacial podrá incorporarse como extensión si el dominio lo justifica.

Entregable: scripts de creación de índices y breve análisis de su utilidad.

5.6. Consistencia y transacciones

El equipo deberá identificar al menos una operación de negocio en la que resulte importante mantener la consistencia. Ejemplos: registrar un pedido y actualizar el stock; registrar un préstamo y cambiar la disponibilidad de un ejemplar; confirmar una inscripción y actualizar las vacantes.

Se deberá explicar qué podría ocurrir si una parte de la operación se completa y otra falla, y proponer una solución mediante una de estas alternativas:

- Operación atómica sobre un documento, cuando el diseño permita resolver correctamente la consistencia en un solo documento.
- Transacción multidocumento, cuando sea necesario modificar varios documentos y las garantías requeridas no puedan resolverse adecuadamente con una operación atómica.

Las transacciones multidocumento requieren un entorno compatible, como un replica set o un clúster apropiado. No debe suponerse que cualquier instancia local independiente admite transacciones. No se exige usar transacciones en todas las operaciones: se evaluará la justificación técnica.

Entregable: descripción del caso, solución implementada y prueba del comportamiento esperado ante errores.

5.7. Análisis de datos

A partir de los datos almacenados, el equipo deberá desarrollar un pequeño módulo de análisis. Se recomienda Python con JupyterLab o Jupyter Notebook, PyMongo, Pandas y Matplotlib. Se admiten otras herramientas justificadas.

- Conexión a MongoDB y recuperación de datos.
- Preparación de los datos necesarios para el análisis.
- Al menos tres indicadores o métricas relevantes.
- Al menos dos visualizaciones diferentes.
- Interpretación escrita de los resultados.
- Conclusión que vincule los resultados con una decisión o recomendación.

Dominio	Indicadores posibles
Productos	Stock por categoría, precio promedio, productos con mayor rotación.
Pedidos	Importe por período, pedidos por estado, ticket promedio.
Cursos	Inscriptos por curso, ocupación, evolución de inscripciones.
Biblioteca	Préstamos por período, libros más solicitados, demora promedio.

Eventos	Participación por actividad, ocupación de sedes, inscripciones por fecha.

Los indicadores deben calcularse a partir de los datos de la base del proyecto. No alcanza con presentar gráficos sin explicar qué información aportan.

Entregable: notebook o módulo ejecutable, visualizaciones y conclusiones.

5.8. Ejecución con Docker Compose

- Configurar un servicio de MongoDB.
- Configurar persistencia mediante un volumen.
- Incluir las variables de configuración necesarias y un archivo de ejemplo sin credenciales reales.
- Documentar cómo iniciar y detener el entorno.
- Documentar cómo cargar los datos de prueba.

Se recomienda incluir la API como servicio en el mismo entorno. En ese caso, la API deberá conectarse a MongoDB utilizando el nombre del servicio de Compose y no localhost dentro del contenedor.

Entregable: docker-compose.yml, configuración y documentación de ejecución.

6. Plan de trabajo y entregas parciales

Semana 1 — Análisis, diseño y modelo documental

- Seleccionar y obtener aprobación del dominio.
- Definir requerimientos y casos de uso.
- Diseñar colecciones y documentos; justificar documentos embebidos y referencias.
- Preparar el entorno MongoDB.

Entrega parcial 1: propuesta y diseño de la base de datos.

Semana 2 — Implementación de MongoDB y consultas

- Crear colecciones y cargar datos.
- Desarrollar las 10 consultas y las agregaciones.
- Crear índices y realizar primeras pruebas de consistencia.

Entrega parcial 2: base poblada, scripts e implementación inicial de consultas.

Semana 3 — API, repositorios y consistencia

- Implementar operaciones CRUD.
- Desarrollar repositorios, validaciones y manejo de errores.
- Implementar y probar el caso de consistencia.

Entrega parcial 3: API conectada a MongoDB y operaciones verificadas.

Semana 4 — Análisis, integración y presentación

- Desarrollar indicadores y visualizaciones.
- Integrar la solución con Docker Compose.
- Probar la ejecución desde un entorno limpio.

- Completar documentación y preparar la defensa.

Entrega final: repositorio completo, documentación y demostración.

7. Entregables finales

- **Informe técnico:** Problema, requerimientos, modelo documental, decisiones de diseño, índices, consistencia, consultas y conclusiones.
- **Scripts y datos:** Creación de colecciones, carga inicial, consultas, índices y datos necesarios para reproducir las pruebas.
- **Código de la aplicación:** API, capa de repositorios, validaciones y pruebas.
- **Análisis de datos:** Notebook o módulo ejecutable, métricas, gráficos e interpretación.
- **Despliegue:** Docker Compose, configuración de ejemplo e instrucciones para instalar y ejecutar.
- **Presentación y demostración:** Exposición breve del problema, decisiones técnicas y demostración funcional.

El código deberá entregarse en un repositorio organizado, preferentemente Git. El README deberá indicar requisitos, comandos de inicio, cómo cargar los datos y cómo ejecutar las pruebas. No se deberán incluir contraseñas reales, tokens ni otros secretos.

8. Criterios de evaluación

Criterio	Puntos	Aspectos evaluados
Diseño documental y justificación	15	Colecciones, documentos anidados, referencias y coherencia con los requerimientos.
Implementación y calidad de los datos	10	Carga reproducible, variedad, validaciones y casos límite.
Consultas y agregaciones	15	Correctitud, variedad, utilidad y complejidad.
API y repositorios	20	CRUD funcional, separación de responsabilidades, errores y validación.
Índices y consistencia	10	Índices justificados, interpretación de explain() y solución del caso de consistencia.
Análisis de datos	10	Indicadores, gráficos, interpretación y conclusiones.
Docker, reproducibilidad y pruebas	10	Entorno funcional desde cero, persistencia y pruebas verificables.
Documentación y defensa individual	10	Claridad de la entrega, demostración y comprensión de las decisiones.

TOTAL	100	

Cada integrante deberá poder explicar el modelo documental, ejecutar una consulta, justificar un índice y describir el funcionamiento del repositorio. La calificación grupal podrá complementarse con una instancia individual de defensa.

9. Alcance y restricciones

- Una problemática por equipo y una entidad principal para el CRUD.
- No se exige una interfaz gráfica completa, autenticación avanzada ni arquitectura de microservicios.
- El análisis debe vincularse con preguntas del dominio y no limitarse a mostrar gráficos.
- Las transacciones multidocumento se implementarán cuando el caso las justifique y el entorno lo permita.
- Se priorizarán tecnologías conocidas por el equipo, siempre que se cumplan los requerimientos.
- Las extensiones avanzadas son opcionales y no reemplazan los requisitos obligatorios.

10. Extensiones opcionales

- Búsqueda de texto o consultas geoespaciales.
- Validación de esquemas mediante reglas de MongoDB.
- Pruebas automatizadas de integración.
- Comparación razonada con una alternativa relacional.
- Tablero interactivo de análisis.
- Pruebas de concurrencia sobre una operación de negocio.

11. Resultado esperado

Al finalizar las cuatro semanas, cada equipo habrá construido una solución operativa pequeña, pero completa: una base documental con datos representativos, una API con repositorios, consultas y agregaciones, índices justificados, un caso de consistencia, un módulo de análisis y un entorno reproducible con Docker Compose.

El propósito central es que los estudiantes puedan diseñar, implementar, consultar y evaluar una base de datos NoSQL a partir de los requerimientos de una aplicación, reconociendo tanto las ventajas del modelo documental como sus compromisos y limitaciones.

Importante: cualquier cambio de alcance o tecnología deberá acordarse con el docente antes de la entrega final.